UGV: Ground Twin of the UAV

Jnanesh Sathisha Karmar, Vivek K Kumar,
Dr. G.V.V. Sharma
Department of Electrical Engineering,
Indian Institute of Technology Hyderabad,
Kandi, India 502284
gadepall@ee.iith.ac.in

***Abstract*—In this paper, we integrate various components of an unmanned air vehicle (UAV) into an unmanned ground vehicle (UGV) to develop a unified control system for both platforms. In particular, we port the battery, communication module, flight controller and GPS from the UAV to the UGV and test their functionality in an outdoor environment. In the process, we obtain an analog twin for the UAV, which allows for extensive testing of various components, without risk of damaging the UAV.**

I. Introduction

Recent advancements in autonomous systems emphasize the development of unified, cross-domain control architectures. While open-source flight stacks like ArduPilot [] and Pixhawk [] are ubiquitous in Unmanned Aerial Vehicles (UAVs), their robust navigation and telemetry algorithms are increasingly being adapted to retrofit off-the-shelf ground chassis into autonomous Unmanned Ground Vehicles (UGVs) [1]. Leveraging a shared architectural ecosystem not only reduces development overhead but also facilitates seamless heterogeneous collaboration, allowing UAVs and UGVs to operate within the same centralized mission framework [2]. However, directly translating a flight-oriented architecture to a ground-based platform introduces significant hardware-level challenges. Specifically, the high-frequency PWM signals designed for aerial Electronic Speed Controllers (ESCs) must be computationally translated into logic-level inputs suitable for standard UGV motor drivers [3]. Our work addresses this gap by detailing the exact middleware translation required to bridge these two domains.

II. System Overview

Fig 1 lists the various components of a drone (UGV) platform based on ArduPilot. To successfully transition from flight to ground, the components shown in Fig 1 must be rerouted. Fig. 2 is the final, unified UGV architecture showing the new data and power workflows, utilizing the ArduPilot, an ESP32 translator, an LM2596 buck converter, and L298N drivers. Table I includes a comperhensive list of all components required for UAV-UGV transition.

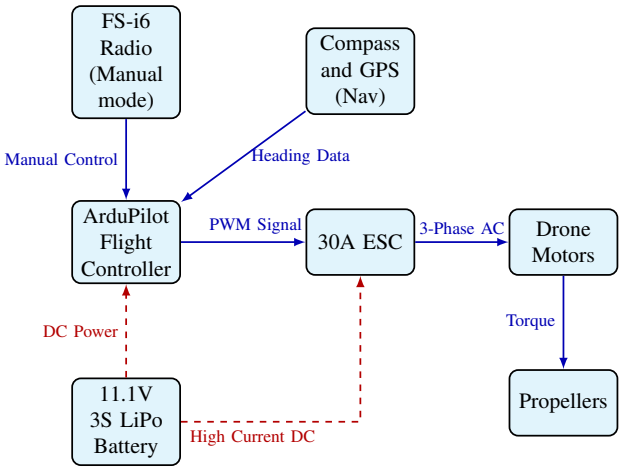

Fig. 1: Original Drone Architecture

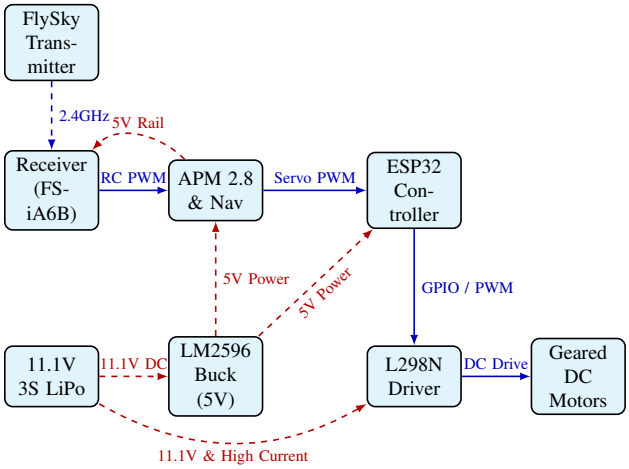

Fig. 2: Integrated UGV Ground Twin Architecture

TABLE I: Component List

Component	Domain	Primary Function
APM 2.8 Flight Controller	Software / Nav	Processes RC inputs and manages GPS/telemetry
ESP32 Microcontroller	Control	Middleware translator and skid steering mixer
L298N Motor Driver	Control	Regulates high-current delivery to the wheels
Geared DC Motors (x4)	Mobility	Replaces brushless air motors for ground traction
11.1V 3S LiPo Battery	Power	Primary high-capacity power source
LM2596 Buck Converter	Power	Steps down 11.1V to a stable 5V logic rail
FlySky FS-i6 Radio	Communication	2.4 GHz wireless manual transmitter
FlySky FS-iA6B Receiver	Communication	Receives wireless inputs and outputs RC PWM
GPS & Compass Module	Software / Nav	Provides heading and location data to the APM

A. *Power*

The same DC power source can be used for both frameworks. The 3S LiPo batteries used in the system are highly versatile. In the original UAV architecture, power distribution relied heavily on SimonK 30A BLDC Electronic Speed Controllers (ESCs) [], which routed high current directly to brushless motors while often supplying low voltage to the flight controller via a built-in Battery Eliminator Circuit (BEC) or a dedicated UAV Power Module.

Transforming this for a UGV required a complete power system overhaul. We discarded the SimonK ESCs (as we used different motors here). Instead, the 11.1V output from the 3S LiPo battery is split into two distinct paths as shown in Fig. **??**. The first path supplies raw 11.1V power directly to the L298N motor driver to satisfy the high-current demands of the geared ground motors. For the second path, we integrated an LM2596 buck converter to step the 11.1V down to a highly stable 5V rail. We used $2000\mu F$ decoupling capacitors. This 5V line is fed directly into the 5V/VIN pin of the ESP32 and is also used to power the APM 2.8 flight controller and its attached peripherals (such as the GPS module). Crucially, because the LM2596 shares a single ground rail, we established a strict common ground connecting the 3S LiPo, the buck converter, the ESP32, the L298N, and the APM 2.8 to prevent logic level drifting and signal errors across the vehicle.

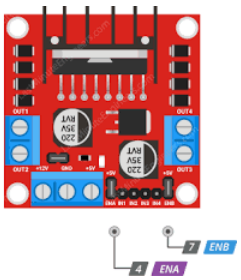

Fig. 3: L298N Motor Driver

B. *Communication:Flysky Radio System*

The Flysky FS-i6 [4] is the transceiver used in most elementary UAV systems. It converts physical inputs from the joystick into digital signals, which are transmitted using the AFHDS 2A (Automatic Frequency Hopping Digital System) protocol. The receiver connected to the robot or vehicle interprets these signals and generates corresponding control signals to motors, servos, or microcontrollers. Because the ground vehicle operates with reduced physical degrees of freedom, the control inputs are drastically simplified. When flying a drone, the system requires continuous input across at least four to six radio channels to manage complex multi-axis flight dynamics and flight modes. For the UGV twin, these multi-axis commands are reduced to just two essential channels: throttle (forward/reverse speed) and steering (left/right turning). The remaining channels provided by the FS-i6 radio are effectively bypassed, streamlining the overall control logic before it even reaches the motor drivers. We have tested the radio system for up to 70m.

Fig. 4: FlySky Remote

III. Mobiity

A. *Degrees of Freedom*

Transitioning from a UAV to a UGV fundamentally alters the system's degrees of freedom (DoF). A drone operates in three-dimensional space, requiring complex physical control over pitch, roll, yaw, and altitude. A ground twin, however, is constrained to a 2D planar surface. This physical limitation inherently removes multiple degrees of freedom, simplifying the mobility dynamics entirely to planar translation and rotation.

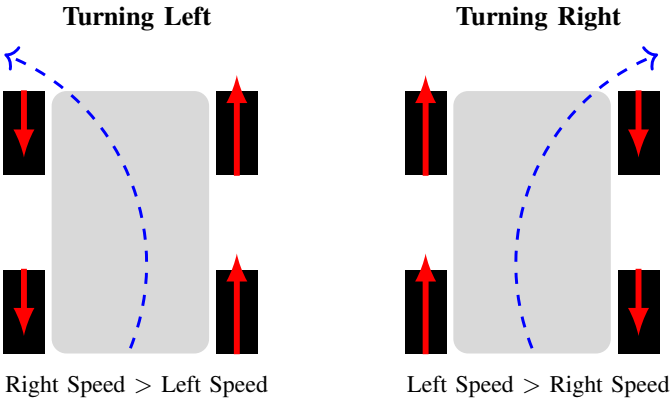

Fig. 5: Skid Steering Visualization

Because of this planar constraint, the UGV employs a skid steering (differential drive) physical configuration (see Fig. 5). Instead of using a complex mechanical steering rack like a traditional car, the vehicle turns by varying the speed and direction of the left and right motor banks independently. A forward command drives all wheels equally, while a steering command creates differential thrust—driving one side faster or in reverse relative to the other—allowing the vehicle to pivot in place or carve turns efficiently.

B. Integration with L298N Motor Drivers

The system architecture follows a sequential control flow from the APM 2.8 flight controller, through an ESP32 microcontroller, to an L298N motor driver [5], which ultimately powers the DC motors.

1) The Signal Translation Challenge: A direct connection between the APM and the motor driver is not feasible due to a fundamental signal mismatch. Specifically, the APM 2.8 outputs standard RC servo PWM pulses ranging from 1000 μs to 2000 μs, whereas the L298N driver requires logic-level directional inputs (HIGH/LOW) and standard duty-cycle PWM (0–100%) to regulate speed.

This highlights a major control paradigm shift between domains. In a UAV, the APM sends high-frequency PWM signals directly to an ESC, which inherently understands those pulses and converts them into 3-phase AC for brushless flight motors. To replicate this control authority in a UGV using standard DC motors, we had to introduce middleware.

2) ESP32 as a Middleware Controller: To bridge this gap, the ESP32 functions as a dedicated signal translator and skid steering mixer. Since the ArduPilot simply passes through raw throttle and steering channel outputs, the ESP32 continuously reads these incoming pulse widths and maps them into directional commands (e.g., 1000–1450 µs for reverse, 1450–1550 µs as a neutral deadband, and 1550–2000 µs for forward). Concurrently, the middleware mathematically combines these two signals to calculate proportional PWM duty cycles for the left and right motor banks. For example, a simultaneous forward and right-steering input prompts the code to increase the left L298N channels while reducing the right, seamlessly executing precise differential drive locomotion.

IV. Software

A. ArduPilot & Mission Planner

ArduPilot [6] is a versatile autopilot system for multicopters, rovers, boats, and planes. It reads sensors (gyro, GPS) to controls motors. One of its key advantages is its ability to handle both high-speed flight stabilization and precise ground steering. This enables the same hardware to be used both as a flight controller and a ground vehicle controller.

Mission Planner [7] is a free, open-source Ground Control Station (GCS) software used to configure, monitor, and control autonomous vehicles such as drones, planes, rovers, and boats that run on ArduPilot firmware. It provides a graphical interface to communicate with the flight controller through USB or telemetry radios. Mission Planner allows users to upload flight missions, set waypoints, tune parameters, calibrate sensors, and monitor real-time flight data such as altitude, speed, GPS position, and battery status.

Fig. 6: Mission Planner interface

B. Setup and Transformation

A major advantage of this architecture is that the software framework requires almost zero custom rewriting; it merely requires a shift in firmware branches. Rather than loading the `ArduCopter` firmware used for multi-axis flight stabilization, we transform the APM into a UGV brain simply by installing ground-specific firmware.

Specifically, flashing the ArduRover firmware transitions the APM into a UGV controller by disabling complex 3D flight stabilization algorithms. In this architecture, the APM software acts as a streamlined passthrough—it simply processes the two active radio channels (throttle and steering) and outputs those distinct signals directly to the ESP32 for downstream processing. See Appendix A for details on code execution.

V. Conclusions and Future Work

We have hence successfully translated the entire UAV architecture into an equivalent UGV architecture across Power, Control, Communication, Mobility, and Software domains.

References

[1] C. L. Darweesh *et al.*, "Initial prototyping of a low-cost unoccupied ground vehicle platform for crop problem risk and severity mapping in agricultural fields," in *Autonomous Air and Ground Sensing Systems for Agricultural Optimization and Phenotyping IX*, vol. 13475. SPIE, 2024, p. 134750I, demonstrates the direct application of an ArduPilot Pixhawk flight controller to modify an RC car into a functional UGV.

[2] L. Castro *et al.*, "Heterogeneous multi-robot collaboration for coverage path planning in partially known dynamic environments," *Machines*, vol. 12, no. 3, p. 200, 2024, highlights the necessity of shared control architectures for UAV and UGV cooperation.

[3] G. Silva, "Implementation of an autopilot for uav/ugv systems based on android smartphone," Master's thesis, Politecnico di Torino, 2019, discusses the integration of autopilot logic, middleware, and standard microcontroller hardware for cross-domain vehicles.

[4] FlySky RC, "Flysky radio control systems," 2024, accessed: 2024-05-19. [Online]. Available: https://www.flysky-cn.com/

[5] STMicroelectronics, *L298 Dual Full-Bridge Driver Datasheet*, STMicroelectronics, 2000, rev. 13. Accessed: 2026-03-29. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[6] ArduPilot Development Team, "Ardupilot: Open source autopilot software," 2024, accessed: 2024-05-19. [Online]. Available: https://ardupilot.org/

[7] M. Oborne and ArduPilot Development Team, "Mission planner: Ground control station," 2024, accessed: 2024-05-19. [Online]. Available: https://ardupilot.org/planner/

APPENDIX

1) Open Mission Planner (you might have to use an older version for better compatibility) and flash the `ArduRover v2.5.1` firmware onto the APM 2.8 flight controller. Custom C++ code is not required for the APM, as the ArduPilot ecosystem provides precompiled stacks in the ArduPilot website.
2) Calibrate radio, compass, GPS, accelerometer and servo output in Mission Planner.
3) Wire APM 2.8 as follows:
   - Inputs: Connect radio receiver and GPS/Compass module.
   - Outputs: Connect signal lines of 1 and 3 to ESP32.
4) Upload the middleware code into the ESP32 from the following repository:

https://github.com/gadepall/ugv-uav/blob/main/codes/main.ino